Mensageria com Google Cloud Pub/Sub

DSM-G04-PI6-2026-2 · Backend de interações e recomendações

> **O que foi implementado**
>
> As interações dos usuários geram eventos persistidos no PostgreSQL. Um worker publica esses eventos no Google Cloud Pub/Sub e consome a assinatura Pull para invalidar o cache de recomendações do usuário.

Projeto analisado

A implementação foi feita em `C:\p.i\DSM-G04-PI6-2026-2\api`. O backend usa Node.js, Express, TypeScript, Prisma com PostgreSQL, validação Zod, autenticação JWT e Swagger. O aplicativo Flutter não recebeu mudanças desta integração.

Não foi encontrada mensageria existente no fluxo analisado. Os controllers já delegavam a persistência aos models; essa organização foi preservada. As alterações que o usuário já tinha no repositório também foram preservadas.

Motivo da escolha

O ranking de recomendações já considera interações, especialmente likes e dislikes. O cache existente dura dez minutos. A mensageria executa a invalidação desse cache em segundo plano, após a gravação de uma interação, sem exigir uma nova entidade de negócio.

Recurso	Valor utilizado
Projeto Google Cloud	dynamic-transit-389813
Tópico existente	pedidos
Assinatura existente	pedidos-consumidor · Pull

Apesar dos nomes dos recursos, o contrato é de **interações**, não de pedidos. Nenhum recurso, permissão ou faturamento foi criado ou alterado no Google Cloud.

Este documento descreve código efetivamente implementado, substituindo a proposta genérica anterior. A conexão real com o Google Cloud ainda depende da configuração e do teste manual no ambiente do usuário.

1. Fluxo implementado

> **POST /interacoes → transação PostgreSQL**
>
> **interação + outbox → worker publicador**
>
> **tópico pedidos → assinatura pedidos-consumidor**
>
> **worker consumidor → invalidação do cache → ack**

Produção e publicação

1. O controller mantém a autenticação, a validação e a resposta HTTP já existentes.
2. Com `PUBSUB_ATIVO=true`, o model grava a interação e o evento na mesma transação. Se a gravação do evento falhar, a interação também é revertida.
3. A resposta HTTP não espera o envio à nuvem. O evento fica na tabela de outbox até o worker conseguir publicá-lo.
4. O worker reserva uma pendência por 120 segundos, aguarda a confirmação do Pub/Sub e salva o identificador da mensagem e a data de publicação.
5. Falhas provocam reagendamento com espera exponencial limitada a cinco minutos. A reserva expira após uma interrupção do processo.

Consumo e processamento

1. O consumidor Pull valida o JSON, o tamanho, o tipo, a versão e o identificador do evento.
2. Consulta a interação no banco para determinar o usuário correto.
3. Na mesma transação, registra o identificador único do evento e remove o cache de recomendações do usuário.
4. Envia `ack` somente depois que a transação termina com sucesso.

Duplicatas e falhas

Uma entrega repetida encontra o identificador já processado e recebe ack sem repetir a invalidação. Essa proteção é persistente e também funciona entre reinicializações e consumidores concorrentes.

Falhas temporárias recebem `nack`. JSON ou contrato inválido, tamanho acima de 64 KiB ou interação inexistente são preservados em quarentena antes do ack. Se a quarentena estiver indisponível, a mensagem recebe nack e pode ser entregue novamente.

O consumidor não publica eventos: isso evita um ciclo de publicação e consumo.

2. Arquivos e dependência

Todos os caminhos abaixo são relativos à pasta `api/`.

Arquivo	Finalidade
`src/mensageria/configuracao.ts`	Validação Zod das variáveis e chave de ativação.
`src/mensageria/contrato.ts`	Contrato versionado, criação do evento e erro de mensagem inválida.
`src/mensageria/cliente.ts`	Cliente oficial compartilhado pelo worker, publicação e fechamento.
`src/mensageria/publicador.ts`	Confirmação do envio e reagendamento da outbox.
`src/mensageria/consumidor.ts`	Validação, ack/nack e encaminhamento à quarentena.
`src/mensageria/persistencia.ts`	Transações, idempotência, reserva da outbox e quarentena no Prisma.
`src/workers/mensageria.ts`	Processo contínuo de publicação e consumo; controle de concorrência e encerramento.
`src/mensageria/mensageria.test.ts`	15 testes isolados, sem banco e sem Google Cloud.
`src/mensageria/persistencia.integration.test.ts`	Teste de transações e concorrência em banco descartável.
`src/models/interacoes.ts`	Grava interação e outbox na mesma transação quando ativado.
`prisma/schema.prisma`	Modelos das três novas tabelas de mensageria.
`prisma/migrations/20261021120000_mensageria_interacoes/migration.sql`	SQL para criar as novas tabelas e o índice da outbox.
`app.ts`	Valida configuração e encerra o servidor e o Prisma corretamente.
`package.json`	Dependência oficial e scripts do worker e testes.
`.env.example`	Variáveis e valores dos recursos existentes; desativado por padrão.
`README.md`	Documentação da configuração, execução, testes e limitações.

Dependência adicionada

`@google-cloud/pubsub: ^6.3.1`, biblioteca oficial para Node.js. O `package-lock.json` local também foi atualizado pela instalação; ele já é ignorado pela configuração existente do repositório.

Não foram adicionados serviços de outro projeto nem alterados os contratos dos endpoints ou os controllers de classificações e interações.

3. Banco e contrato do evento

Tabela nova	Conteúdo e função
tb_evento_outbox	id_evento (chave primária), conteudo JSON, criado_em, publicado_em, id_mensagem, tentativas e proxima_tentativa_em. Possui índice por publicação e próxima tentativa.
tb_evento_processado	id_evento (chave primária) e processado_em. Impede efeitos repetidos na mesma transação da invalidação.
tb_mensagem_quarentena	chave, id_mensagem, conteudo binário, motivo e criado_em. A chave usa hash da mensagem para não duplicar o registro de uma mesma entrega inválida.

A migration adiciona essas tabelas; não recria as entidades de negócio. Antes de aplicar `prisma migrate deploy`, revise também quaisquer migrations anteriores pendentes no seu ambiente.

Exemplo de mensagem real

```
{
  "id_evento": "interacao:42",
  "tipo": "interacao.registrada",
  "versao": 1,
  "ocorrido_em": "2026-10-21T12:00:00.000Z",
  "dados": { "id_interacao": 42 }
}
```

O exemplo só pode ser processado se a interação 42 existir no mesmo banco do worker. O ID estável é sempre `interacao:<id_interacao>`. A validação exige ID positivo compatível com o inteiro do PostgreSQL, data ISO, versão 1 e tipo correto. Campos extras são rejeitados.

Na publicação, os atributos são `tipo=interacao.registrada` e `versao=1`. A mensagem não carrega senha, token, e-mail ou os dados completos do usuário.

Consultas para acompanhar

```
SELECT id_evento, publicado_em, tentativas,
       proxima_tentativa_em
FROM tb_evento_outbox;

SELECT id_evento, processado_em
FROM tb_evento_processado;

SELECT chave, id_mensagem, motivo, criado_em
FROM tb_mensagem_quarentena;
```

O conteúdo bruto da quarentena pode ter dados enviados indevidamente por um produtor. Restrinja seu acesso e defina uma política de retenção.

4. Configuração e execução

Acrescente os valores ao `api/.env`, preservando as configurações atuais do banco e JWT. O arquivo real não foi alterado pela implementação.

```
PUBSUB_ATIVO=true
GOOGLE_CLOUD_PROJECT=dynamic-transit-389813
PUBSUB_TOPICO=pedidos
PUBSUB_ASSINATURA=pedidos-consumidor
PUBSUB_CONCORRENCIA=5
```

O exemplo vem com `PUBSUB_ATIVO=false`. A concorrência permitida é de 1 a 100 mensagens, com padrão 5. API e worker devem usar o mesmo banco. O worker não precisa do segredo JWT.

Autenticação local por ADC

```
gcloud auth application-default login
gcloud auth application-default set-quota-project dynamic-transit-389813
```

Na hospedagem, use a identidade do ambiente de execução. Nenhuma chave JSON deve ser gerada ou incluída no código. A identidade precisa publicar no tópico (`roles/pubsub.publisher`) e consumir a assinatura (`roles/pubsub.subscriber`), além de acessar o banco. Confirme a habilitação da API Pub/Sub e eventual permissão de uso do projeto de cota.

Preparar e iniciar a API — PowerShell

```
cd C:\p.i\DSM-G04-PI6-2026-2\api
npm install
npm run prisma:generate
npm run prisma:migrate
npm run build
npm start
```

Iniciar o consumidor e publicador — outro terminal

```
cd C:\p.i\DSM-G04-PI6-2026-2\api
npm run worker:start
```

Em desenvolvimento, use `npm run dev` para a API e `npm run worker:dev` para o worker, em terminais separados.

O worker precisa de hospedagem com processo contínuo e reinício automático. Uma função HTTP efêmera não mantém o consumidor Pull. Não foi encontrada configuração de deploy que garanta esse processo.

SIGINT e SIGTERM iniciam o encerramento: o worker aguarda publicação e tarefas em andamento e fecha Pub/Sub e Prisma. Seu prazo máximo é de 90 segundos; o da API, de 30 segundos.

5. Testes realizados

> **Resultado: 15 testes isolados + 1 teste de integração aprovados.**
>
> Também passaram a compilação TypeScript e a validação do schema Prisma. Nenhuma mensagem foi enviada ao Google Cloud durante os testes.

Cobertura dos 15 testes isolados

1. Ack somente após concluir o processamento.
2. JSON inválido vai para quarentena antes do ack.
3. Contrato inválido não executa a regra de negócio.
4. Falha temporária produz nack.
5. Falha da quarentena permite nova tentativa.
6. Interação inexistente recebe tratamento permanente explícito.
7. Entrega duplicada é confirmada sem novo efeito.
8. ID do evento deve corresponder à interação.
9. Versão e identificadores inválidos são rejeitados.
10. Mensagem acima de 64 KiB recebe tratamento explícito.
11. Publicação aguarda confirmação antes de marcar envio.
12. Falha de envio mantém o ID e reagenda com espera.
13. Falha ao marcar envio permite republicação do mesmo evento.
14. Outbox vazia não provoca envio.
15. Ativação exige os recursos e respeita os padrões de configuração.

Integração com PostgreSQL 18

Foi iniciado um servidor temporário isolado na máquina, sem usar o banco do projeto. As quatro migrations foram aplicadas do zero. O teste verificou gravação da outbox, duplicatas concorrentes, reserva concorrente, ausência de novo efeito após repetição, rollback em falhas e persistência idempotente da quarentena. Ao terminar, o servidor foi encerrado e removido.

Como repetir

```
npm test
npm run test:integration
```

O segundo comando é ignorado sem `DATABASE_URL_TESTE`. Para habilitá-lo, prepare um PostgreSQL descartável e vazio, aplique as migrations nele e configure essa variável com sua URL. Não use o banco real nem produção. Os testes não publicam eventos no Pub/Sub.

A auditoria `npm audit --omit=dev` registrou 9 alertas nas dependências de produção: 3 moderados, 5 altos e 1 crítico. A lista não incluiu `@google-cloud/pubsub`. Atualizações gerais dessas dependências não foram feitas nesta integração.

6. Teste manual e pendências

Verificar envio e recebimento

1. Em ambiente de desenvolvimento autorizado, aplique as migrations, configure ADC e ative a mensageria. Inicie API e worker.
2. Autentique-se pelo fluxo existente em `/auth/login`. Use o token JWT no Swagger em `/docs`.
3. Escolha um estabelecimento existente e envie `POST /interacoes` com `id_estabelecimento` válido e `tipo_evento` igual a `like` ou `dislike`.
4. A resposta 201 inclui o ID da interação. Localize `interacao:<id>` na outbox. O worker deve registrar os estados `publicado` e `processado`.
5. Confira o identificador em `tb_evento_processado`. Se havia cache do usuário, ele deve ser removido; a próxima consulta de recomendações recalcula o ranking.
6. Para simular duplicata em desenvolvimento, pare o worker e altere apenas a linha de teste da outbox: `publicado_em=NULL` e `proxima_tentativa_em=CURRENT_TIMESTAMP`. Reinicie: o consumidor deve indicar `duplicado` sem novo efeito.
7. Para observar recuperação, registre uma interação com o worker parado. Reinicie-o e acompanhe a publicação da pendência. Testes de indisponibilidade de rede devem ser feitos apenas em ambiente isolado.

O que ainda depende do ambiente

Aplicar a nova migration no banco real, configurar ADC e as permissões, verificar a ligação entre tópico e assinatura e iniciar o worker contínuo. Nenhuma dessas alterações externas foi feita automaticamente. Não foi realizado teste de ponta a ponta com o Google Cloud.

Limitações e cuidados operacionais

A invalidação é eventual. Até o processamento, uma resposta pode usar o cache antigo; uma consulta concorrente pode voltar a preenchê-lo. O TTL anterior de dez minutos continua existindo. Não há promessa de consistência forte ou de entrega exatamente uma vez de ponta a ponta.

Com a integração desativada, o comportamento anterior é mantido e nenhuma outbox é criada. Não existe backfill automático de interações antigas. Preserve os identificadores processados enquanto seus eventos puderem ser repetidos.

Outros consumidores na mesma assinatura competem por mensagens. Contratos antigos ou incompatíveis serão enviados à quarentena: revise o backlog antes de ativar. Monitore pendências antigas, falhas, retentativas e quarentena. Os logs do worker contêm estado e identificador, sem payload ou credenciais.

Referências oficiais:
https://cloud.google.com/pubsub/docs/authentication
https://cloud.google.com/pubsub/docs/access-control
https://googleapis.dev/nodejs/pubsub/latest/